Ejercicios: Internet, HTTP y APIs

1. **Del Cliente al Servidor**

Explique paso a paso qué ocurre cuando usted escribe www.youtube.com en el navegador hasta que el video aparece en pantalla.

- Debe incluir en su explicación:
  - Qué hace el cliente (navegador).
    - Valida la URL (www.youtube.com)
    - Verifica si la IP está en caché (DNS cache)
    - Solicita la IP al DNS si no la tiene
    - Abre una conexión TCP
    - Inicia HTTPS (TLS handshake)
    - Envía una solicitud HTTP (GET)
    - Descarga HTML, CSS, JS, imágenes y video
    - Renderiza el contenido y reproduce el video
  - Qué papel cumple el DNS.
    - Traduce el nombre de dominio (www.youtube.com) a una dirección IP, usando una cadena jerárquica de servidores DNS, puede usar caché para acelerar la respuesta
  - Qué ocurre con la dirección IP.
    - Se usa para:
      1. Establecer una conexión TCP/IP
      2. Identificar el servidor destino
      3. Enviar paquetes de red
  - Qué hace el servidor.
    - Recibe un request HTTP/HTTPS
    - Analiza headers, cookies, tokens
    - Decide qué contenido devolver
    - Envía Response con:
      1. HTML
      2. JSON
      3. Video (streaming)
      4. Status code (200, 302, 404, etc.)

- Cómo entra en juego el protocolo HTTP/HTTPS.
    - Define:
        1. Métodos (GET, POST)
        2. Headers
        3. Status codes
    - HTTPS:
        1. Cifra los datos
        2. Protege cookies, tokens y credenciales

2. **Frontend y Backend en acción**

Suponga que está construyendo una app web para agendar citas médicas.

- Indique qué parte del sistema correspondería al frontend y cuál al backend.
    - Toda la parte de interacción con el usuario, formularios, botones, etc. Pertenecen al Front End
    - La parte de Bases de Datos y estructura para guardar las citas agendadas, Autenticación y autorización (login, roles), Reglas de negocio (no permitir citas duplicadas, horarios disponibles, etc.), Validaciones y seguridad, corresponden al Back End, así como el API para comunicar ambas
- Mencione tres tecnologías posibles para cada uno.
    - Front End: Javascript, CSS, HTML
    - Back End: Python, Node, Java.
    - BD: SQL, MySQL, PostgreSQL, MongoDB
- Explique brevemente cómo el frontend se comunicaría con el backend (mencione los conceptos de API, HTTP y request/response).
    - El FE hace un HTTP request a un endpoint de una API, la API recibe el request en el servidor, el backend ejecuta lógica + consulta/guarda en la BD, devuelve un HTTP response con: status code (200, 201, 400, 401, 500…), datos (normalmente JSON) o un error

3. **REST vs SOAP vs GraphQL**

Comparar estas opciones según formato de datos, flexibilidad, dificultad de implementación y uso actual, para entender cuándo conviene cada una.

Cómo completar la tabla:

- ✓ Usa términos concisos (p. ej., “JSON/XML” en formato de datos).
- ✓ En “nivel de flexibilidad”, piensa: ¿quién controla mejor lo que se envía/recibe, cliente o servidor?
- ✓ En “dificultad de implementación”, considera curva de aprendizaje, herramientas y mantenimiento.
- ✓ En “uso actual”, valora su presencia en proyectos modernos (Alta/Media/Baja).

Complete la siguiente tabla:

Tipo de API	Formato de datos usado	Nivel de flexibilidad	Dificultad de implementación	Uso actual (Alta / Media / Baja)
REST	**JSON(Comun) / XML**	Medio / Alto (el servidor define endpoints; el cliente elige filtros/params)	Facil / Medio	Alto
SOAP	**XML**	Bajo (contrato rígido; control mayor del servidor/contrato)	Alto	Bajo / Medio
GraphQL	**JSON(Queries/ Mutations)**	Muy alto (el cliente decide exactamente qué campos recibe)	Medio / Alto	Medio / Alto

Luego conteste:

¿Cuál considera más apropiada para una startup moderna que desarrolla un sistema de reservas en línea? ¿Por qué?

- REST ya que es más simple y rápido de implementar y mantener, tiene mejor soporte universal (caching HTTP, proxies, monitoreo, herramientas)

4. **Explorando APIs con Postman**

Parte 1: Selección de la API

1. Busca una API pública gratuita (sin autenticación obligatoria o con token gratuito).

API:

- PokéAPI

2. Escribe en tu documento:
   - PokéAPI
   - PokéAPI es una API pública gratuita que proporciona información detallada sobre el universo Pokémon, incluyendo Pokémon, habilidades, tipos, movimientos y generaciones. No requiere autenticación ni token.

     URL: https://pokeapi.co/api/v2

Parte 2: Configuración en Postman ⚙️

1. Crea una **nueva colección** en Postman con el nombre de la API.
   - Nombre de la colección en Postman: **PokéAPI**
2. Agrega al menos **3 tipos de solicitudes**:
   - **GET** → obtener información (por ejemplo, lista o detalle de un elemento)
     - https://pokeapi.co/api/v2/pokemon - Obtiene una lista paginada de Pokémon.
     - https://pokeapi.co/api/v2/pokemon/pikachu - Obtiene el detalle de un Pokémon específico (Pikachu).
     - https://pokeapi.co/api/v2/type/fire - Obtiene información sobre el tipo “Fire” y los Pokémon asociados.
   - **POST** → enviar datos (por ejemplo, crear un recurso).
     - No permite el crear nuevos recursos
   - **PUT/PATCH o DELETE** → actualizar o eliminar un recurso (si la API lo permite).
     - No permite el crear nuevos recursos

3. Si la API usa **parámetros** o **tokens**, configura las variables necesarias en la pestaña **Environment**.
    1. No se requiere configuración de:
        - Tokens
        - Headers de autenticación
        - Variables de entorno

Parte 3: Ejecución y análisis

1. **Ejemplo de resultados:**
    - **Solicitud GET /pokemon:**
        - **Status Code: 200 OK**
        - **Content-Type: application/json**
        - **Body: Lista de Pokémon con name y url**
    - **Solicitud GET /pokemon/pikachu:**
        - **Status Code: 200 OK**
        - **Body: JSON con estadísticas, habilidades, tipos y sprites**
    - **Solicitud GET /type/fire:**
        - **Status Code: 200 OK**
        - **Body: Lista de Pokémon tipo fuego**
    - **Solicitud GET /pokemon:**
        - **Status Code: 200 OK**
        - **Content-Type: application/json**
        - **Body: Lista de Pokémon con name y url**
    - **Solicitud GET /pokemon/pikachu:**
        - **Status Code: 200 OK**
        - **Body: JSON con estadísticas, habilidades, tipos y sprites**
    - **Solicitud GET /type/fire**
        - **Status Code: 200 OK**
        - **Body: Lista de Pokémon tipo fuego**

Parte 4: Explicación técnica (README.md)

Descripción general

PokéAPI es una API REST que expone recursos relacionados con Pokémon mediante endpoints HTTP accesibles usando solicitudes GET y respuestas en formato JSON.

- **Solicitud 1: Lista de Pokémon**
  - **Método: GET**
  - **Endpoint: /pokemon**
  - **Parámetros: ninguno**
  - **Respuesta: lista paginada de Pokémon**


```
{
  "count": 1302,
  "results": [
    { "name": "bulbasaur", "url": "..." },
    { "name": "ivysaur", "url": "..." }
  ]
}
```


- **Solicitud 2: Detalle de Pokémon**
  - **Método: GET**
  - **Endpoint: /pokemon/pikachu**
  - **Respuesta: información detallada del Pokémon**


```
{
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "types": [
    { "type": { "name": "electric" } }
```

```
  ]
}
```

- **Solicitud 3: Tipo Fire**
  - **Método: GET**
  - **Endpoint: /type/fire**
  - **Respuesta: Pokémon asociados al tipo fuego**

Lo aprendido

- **Cómo estructurar requests HTTP**
- **Cómo interpretar respuestas JSON**
- **Uso de status codes**
- **Navegación entre recursos relacionados usando URLs**

Parte 5: Reflexión final

¿Qué aprendí sobre las APIs?

Aprendí que las APIs permiten la comunicación estructurada entre cliente y servidor mediante protocolos HTTP, usando endpoints claros y respuestas estandarizadas como JSON. También comprendí cómo los recursos se relacionan entre sí y cómo navegar esos datos.

¿Cómo ayudó Postman?

Postman facilitó la comprensión del ciclo request/response, permitiendo visualizar fácilmente status codes, headers y cuerpos de respuesta sin necesidad de escribir código. Esto ayuda a depurar y entender el comportamiento de una API antes de integrarla en una aplicación real.

Nota:

PokéAPI es una API REST de solo lectura, por lo que no admite POST, PUT o DELETE.